Smart Banking Management System — Project Specification

Oct 8, 2026 · @Abdullah

1. Project overview

Build a console-based Smart Banking Management System in Core Java that manages customers, bank accounts, transactions, loans and employees.

Problem statement

The bank currently manages customer information and transactions manually. This makes it hard to:

- Track account balances
- Calculate interest
- Manage multiple account types
- Maintain transaction history

Constraints

Item	Requirement
Language	Core Java
Interface	Console-based application
Users	Bank staff (Tellers, Managers, Administrators)
Persistence	All data persists between runs, stored in CSV files
Extensibility	New account types, transaction types and banking features can be added without significantly modifying existing code

Modules

1. Customer Management

2. Bank Account Management
3. Transaction Management
4. Employee and Role Management
5. Notifications
6. Loan Management

2. Module 1: Customer Management

Bank staff register and maintain customer records; one customer can own many accounts.

Functions

1. Register a new customer
2. Update customer information
3. Search for customers by ID or name
4. View customer details
5. View all accounts belonging to a customer
6. Deactivate a customer account

Customer data

- Customer ID (unique)
- Full name
- Email
- Phone number
- Address
- Registration date

Business rules

- A customer can have multiple bank accounts.
- Duplicate customer IDs must not be allowed.
- A deactivated customer cannot open new accounts.
- Customer information must be accessible only through controlled methods.

3. Module 2: Bank Account Management

The bank offers three account types, each with its own withdrawal and interest rules.

Account types

Account type	Interest	Withdrawals	Other rules
Savings Account	Earns interest	Limited number of withdrawals	Minimum balance requirement
Current Account	No interest	Can go below zero up to the overdraft limit	Supports an overdraft limit
Fixed Deposit Account	Fixed interest rate	Early withdrawals restricted	Has a maturity date

Functions

1. Open an account
2. View account balance
3. Update account status
4. Close an eligible account

Business rules

- Every account must have a unique account number.
- Account balances cannot be modified directly from outside the account's controlled operations.
- Each account type must implement its own withdrawal and interest rules.
- A closed account cannot perform transactions.
- Accounts with outstanding liabilities cannot be closed.
- An account can be closed only when its balance is zero.

4. Module 3: Transaction Management

Every deposit, withdrawal and transfer is validated, applied safely and recorded.

Functions

1. Deposit money
2. Withdraw money
3. Transfer money between accounts
4. View transaction history
5. Search transactions by date or transaction type
6. Generate a mini statement showing the last five transactions

Transaction data

- Transaction ID (unique)
- Source account
- Destination account (where applicable)
- Amount
- Transaction type
- Timestamp
- Status

Business rules

- Negative and zero transaction amounts must be rejected.
- Withdrawals must follow the rules of the relevant account type.
- Transfers must be atomic: either both balances update or neither does.
- Failed transactions must not incorrectly modify balances.
- Every successful financial operation must be recorded.
- Each transfer must use a single unique transaction reference.

5. Module 4: Employee and Role Management

Three employee roles, each allowed a different set of actions; anything outside a role's permissions is rejected.

Roles and permissions

Role	Permissions
Bank Teller	Register customers, open accounts, process deposits and withdrawals
Bank Manager	Approve loans, manage account statuses, view branch reports
Bank Administrator	Manage employees, configure banking rules, access all reports

Requirements

- Every employee must have a unique ID.
- Different roles must have different permissions.
- Unauthorized actions must be rejected.
- The system must keep a record of sensitive administrative actions.

6. Module 5: Notifications

Customers are notified by email after four kinds of events.

Notification events

- Successful deposit
- Successful withdrawal
- Fund transfer
- Account closure

Channel

Email is simulated: the message (recipient, subject, body) is printed to the console instead of being sent.

7. Module 6: Loan Management

A minimal loan flow: staff record an application, a Manager approves or rejects it, and repayments are recorded until it is paid off.

Functions

1. Record a loan application for a customer
2. Approve or reject a loan (Bank Manager only)
3. Record a loan repayment
4. View all loans of a customer

Loan data

- Loan ID (unique)
- Customer ID
- Linked account number
- Principal amount
- Outstanding amount
- Status: Pending, Approved, Rejected or Repaid

Business rules

- Only active customers can apply for a loan.
- The loan amount must be greater than zero.
- Only a Bank Manager can approve or reject a loan.
- On approval, the principal is credited to the linked account and recorded as a transaction.
- Repayments are debited from the linked account and reduce the outstanding amount; at zero the loan becomes Repaid.
- An account linked to a loan that is not yet Repaid has an outstanding liability and cannot be closed.

8. Open questions

The brief leaves these points unspecified; confirm them with the mentor before building the affected parts.

- [ ] **Rule values:** what are the Savings minimum balance and withdrawal limit, the Current overdraft limit, the interest rates, and the Fixed Deposit early-withdrawal rule? Are they fixed, or set by the Administrator under "configure banking rules"?
- [ ] **Employee login:** how does a staff member identify themselves in the console so the system can check their role?
- [ ] **Branch reports:** what should a branch report contain?
- [ ] **Deliverables:** deadline, allowed libraries (e.g. JUnit for tests), and whether a design document or UML diagram is expected.